Universidad Diego Portales

Facultad de Ingeniería y Ciencias

Tarea 2

Torneo de Komikan Concurrente

Asignatura: Sistemas Operativos

Estudiante: Kevin Cornejo

Fecha: Octubre de 2026

1. Introducción

En esta tarea desarrollamos un programa en C que simula un torneo de Komikan, utilizando hilos para que varias partidas puedan funcionar al mismo tiempo.

La idea principal fue controlar el acceso al tablero, ya que las fichas se mueven de forma independiente y podrían intentar ocupar una misma posición.

2. Cómo organizamos los hilos

Para el torneo usamos la biblioteca `pthread`.

Cada jugador tiene un hilo que participa en el emparejamiento. Cuando se juntan dos jugadores, se crea una partida y un hilo que funciona como juez.

Dentro de cada partida se crean 13 hilos: uno para el Puma y doce para los Perros. Cada ficha espera un tiempo aleatorio antes de intentar moverse.

Como no hay turnos, las fichas pueden intentar moverse al mismo tiempo, pero el tablero se modifica de forma controlada.

3. Sincronización

Para evitar problemas al mover las fichas usamos un `mutex` por partida.

Antes de realizar un movimiento, el hilo bloquea el mutex. Luego revisa si la casilla está disponible, mueve la ficha y comprueba si alguien ganó. Al terminar, libera el mutex.

De esta forma evitamos que dos fichas ocupen la misma casilla o que se modifique el tablero mientras otro hilo lo está revisando.

También usamos un semáforo para limitar la cantidad de mesas disponibles. Si todas están ocupadas, las otras partidas deben esperar.

Por último, utilizamos variables de condición para que los jugadores puedan esperar a su pareja o al término de una partida sin estar revisando constantemente.

4. Problemas que podrían ocurrir

Uno de los problemas posibles es el *deadlock*, que ocurre cuando los hilos quedan esperando recursos y ninguno puede avanzar.

Para evitarlo intentamos mantener las secciones críticas cortas y liberar los mutex después de cada movimiento. Además, cada partida tiene su propio mutex, por lo que las fichas de mesas distintas no necesitan bloquearse entre sí.

También puede existir *starvation*, cuando un hilo espera demasiado para conseguir acceso al tablero. En nuestro programa cada ficha espera un tiempo aleatorio antes de moverse, lo que ayuda a repartir los intentos, aunque no garantiza que todos los hilos tengan las mismas oportunidades.

Otro caso es el *livelock*, cuando las fichas siguen moviéndose, pero la partida no logra terminar. Como los movimientos son aleatorios, podría pasar que una partida dure bastante tiempo.

5. Decisiones que tomamos

Decidimos usar hilos en lugar de procesos porque todas las fichas de una partida necesitan compartir el mismo tablero.

Esto permite trabajar directamente con la memoria compartida del programa, sin tener que crear mecanismos adicionales para intercambiar información entre procesos.

El mutex hace que los movimientos se ejecuten uno a la vez dentro de cada partida. Esto puede generar algo de espera, pero permite mantener el tablero correcto.

Por otro lado, las mesas funcionan de manera independiente. Así podemos tener varias partidas activas, siempre respetando el máximo indicado al ejecutar el programa.

Para las interrupciones usamos una señal que permite detener el torneo con `Ctrl+C`. También agregamos una opción para simular el abandono de un jugador y terminar su partida.

6. Conclusión

Con esta tarea pudimos aplicar hilos, mutex, semáforos y variables de condición en un mismo programa.

Lo más importante fue lograr que las fichas se movieran de forma concurrente sin provocar conflictos en el tablero, y controlar que no se utilizaran más mesas de las permitidas.

También comprobamos el funcionamiento con distintas cantidades de jugadores, las dos condiciones de victoria y la interrupción del torneo.